Contents

- Sherlock AI: Project Report
  - Executive summary
  - 1. Project overview
  - 2. Case catalogue
  - 3. System design
  - 4. Investigation features
  - 5. Quality and verification
  - 6. Strengths
  - 7. Limitations and risks
  - 8. Recommended next steps
  - Conclusion
  - Sources and scope

Sherlock AI: Project Report

Report date: 23 October 2026
Purpose: Technical and product overview based on the project repository.

Executive summary

Sherlock AI is a 2D detective game built around a symbolic reasoning engine. Players explore case locations, question witnesses, collect evidence, and inspect how known facts support or weaken possible suspect theories. A React and Phaser client handles the game and investigation workbench; a Python FastAPI service owns case data, session state, and inference. The reasoning is rule based and does not use an LLM to decide case outcomes.

The repository contains six playable case definitions. Together they define 33 rooms, 20 non-player characters, 56 interactable objects, 79 evidence items, and 61 inference rules. The project has a working local Docker Compose setup and CI definitions for backend and frontend checks. The README identifies production deployment and its operating configuration as unfinished, so the project is best described as a feature-rich development build rather than a production release.

1. Project overview

1.1 Product goal

The game teaches evidence-based deduction through play. A player discovers evidence and testimony, sees resulting facts and deductions, and can examine the proof behind a theory before

making a final accusation. The core design distinction is that a *possible suspect* is not automatically treated as *guilty*.

1.2 Intended experience

An investigation moves through a repeatable loop:

1. Select a case and difficulty.
2. Explore connected rooms and interact with objects and characters.
3. Collect clues and testimony as the session progresses.
4. Review evidence, derived facts, timelines, contradictions, and suspect comparisons.
5. Form or test a theory, then make a final accusation.

The game supports keyboard, controller, and touch input. Standard mode offers hints; Classic mode disables them and applies a score multiplier. A daily case mode uses a date-based seed.

2. Case catalogue

Case	Mystery	Rooms	Characters	Evidence	Rules
Blackwood Manor	A diamond disappears from a manor safe.	7	4	13	11
Holloway Theatre	A violin vanishes from a dressing-room safe.	6	4	12	12
Saltmarsh Lighthouse	A beacon goes dark before a shipwreck.	4	3	11	10
Starfall Observatory	A comet negative disappears from a locked dome.	5	3	14	9
The Glasshouse Orchid	A rare orchid is taken from a locked case during a storm.	5	3	14	9
The Midnight Mail	Diplomatic dispatches go missing from a locked mail van.	6	3	15	10
Total		**33**	**20**	**79**	**61**

Each case is represented by eight JSON files covering its metadata, map, entities, evidence, dialogue, background facts, rules, and events. The data-driven format lets the backend validate and load cases consistently while the frontend supplies a corresponding room layout and art.

3. System design

Architecture at a glance: Player → React interface and Phaser game → FastAPI service. The API uses the case loader and validator, SQLite investigation state, and symbolic inference services. It returns facts, proofs, hints, and verdict evidence to the client.

3.1 Frontend

The frontend uses React, TypeScript, Phaser 3, and Vite. React renders the menu, case controls, investigation workspaces, and overlays. Phaser renders the explorable rooms, player, doors, objects, and characters. A typed event bridge connects meaningful game interactions to the React session flow; movement remains local to the game scene.

The investigation workbench groups tools into three tabs:

- **Case File:** evidence notebook, deductions, suspect comparison, and accusation.
- **Reasoning:** chronology, contradictions, what-if analysis, evidence links, and knowledge graph.
- **Field Kit:** photos, world guide, and investigation log.

The interface supports keyboard, gamepad, and touch input, sound controls, and reduced-motion preferences. In-game text labels are shown contextually to reduce clutter; the current room, evidence count, and nearby interaction remain available.

3.2 Backend and persistence

The backend uses FastAPI, Pydantic, SQLAlchemy, and SQLite. It exposes case and session APIs, validates case files at load time, stores investigation sessions and discoveries, and provides health and database-readiness endpoints. Compose stores the SQLite database in a named volume so routine container recreation preserves saves.

The client keeps some convenience data in browser storage, including local notes, tags, photos, settings, and references used to resume server-side sessions. The repository also includes database backup and restore instructions and a browser-data export flow.

3.3 Inference model

The AI component is a symbolic logic system:

- Facts are represented as predicates; rules use variables and conditions.
- Unification matches rule conditions to known facts.
- Forward chaining derives new facts until no further facts can be added.
- Backward chaining attempts to prove a requested goal and returns a proof tree.
- Derived facts retain rule and evidence provenance for explanations.
- A separate resolution-refutation endpoint can cross-check a proof; it is isolated from the normal gameplay inference path.

The case loader checks identifiers and cross-file references, validates rules, and rejects rules that conclude `guilty`. The player knowledge base receives background facts and legitimately discovered evidence, not the case's hidden truth block. Verdict grading checks the accusation against that truth separately and reports supporting and contradicting evidence.

4. Investigation features

The service supports session creation and resumption, room movement events, object interaction, dialogue visits, evidence and knowledge retrieval, suspect accusation, and hints. Reasoning tools

include goal queries with explanations, a session knowledge graph, what-if analysis, contradiction reports, suspect comparisons, evidence-link analysis, and a player-known-facts export.

Other player-facing features include a chronology, world map/guide, investigation log, clue photography, private notebook notes and tags, achievements, scoring, and an optional Forensics memory exercise. These features are presented as workbench tools so the player can move between the game world and case analysis.

5. Quality and verification

The repository contains 23 Python test files with 140 test functions, covering the logic engine, case validation, APIs, and case walkthroughs. CI is configured to run Ruff and pytest for the backend, and TypeScript checking, asset validation, and a production build for the frontend. The asset validator checks texture references across all six layouts and door geometry.

The most recent recorded frontend build completed successfully: asset validation, TypeScript checking, and Vite production build passed. That build emitted a bundle-size warning because the Phaser chunk exceeded 1 MB after minification. This report does not claim a fresh backend test run or live end-to-end playthrough.

6. Strengths

- **Inspectable deductions:** proofs can be traced to rules and source facts instead of presenting unexplained conclusions.
- **Spoiler separation:** the case truth is kept out of ordinary player knowledge and case-summary responses.
- **Reusable content model:** cases are data driven and validated across files before use.
- **Multiple analysis paths:** the player can compare suspects, inspect contradictions, test hypotheticals, and view evidence relationships.
- **Operational basics:** the repository includes Compose startup, persistent local saves, health/readiness checks, and backup/restore guidance.
- **Player accessibility options:** keyboard, controller, touch, audio controls, and reduced-motion support are implemented.

7. Limitations and risks

1. **Production operations are undefined.** The README marks release readiness as in progress and does not specify a deployment target or production operating configuration.
2. **The Phaser bundle is large.** The latest recorded frontend build warns that the engine chunk exceeds 1 MB minified, which can slow first-time gameplay loading on constrained networks.
3. **Gameplay verification is incomplete in the available evidence.** Automated case and API coverage exists, but this report has no recent cross-device playthrough evidence for all six cases.
4. **SQLite is suited to the documented local/single-instance setup.** The repository does not establish a multi-instance production database, migration, or recovery strategy beyond local backup guidance.

5. **The workbench is feature rich.** Workspace tabs and contextual game labels reduce simultaneous text, but usability still needs observation with new players, especially on narrow screens.

8. Recommended next steps

1. **Run structured playtests** across all six cases, with desktop, mobile touch, and gamepad coverage. Track completion, navigation failures, clue discoverability, and time to first deduction.
2. **Set a release target** and document hosting, environment configuration, database choice, backups, monitoring, and update/rollback procedures before calling the project production ready.
3. **Measure first-load performance** on representative devices and networks, then split or defer game-engine code if measurements confirm the bundle warning affects players.
4. **Review case balance and clarity** using observed player progress and the existing loader diagnostics; ensure rules yield useful explanations without making evidence dependencies opaque.
5. **Validate responsive usability and accessibility** with keyboard-only and screen-reader navigation, reduced motion, touch targets, and small viewport testing.

Conclusion

Sherlock AI combines an exploratory mystery game with an explainable, deterministic logic engine. Its six structured cases and investigation workbench provide a strong basis for evidence-led play, and the repository includes substantial validation and local operations support. The next step is to turn that feature set into a dependable release through cross-device playtesting, performance work guided by measurement, and a defined production operating plan.

Sources and scope

This report was prepared from the repository on 23 October 2026. Primary sources include `README.md`, `CASE_AUTHORING.md`, the six directories under `cases/`, `frontend/src/app/App.tsx`, `frontend/src/game/`, `backend/app/`, `backend/tests/`, `.github/workflows/ci.yml`, and `docker-compose.yml`. Feature and test counts above are repository inventory counts, not independent user-study results. No external literature or performance benchmarks were used.

AI assistance disclosure: This report was drafted with AI assistance from the project repository. The repository does not document an independent human review of this report.